

Step 4 — Structured Output and Streaming the Final Response — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below are run from inside `project_13_langchain_patterns/`, continuing from Step 3's `handlers.py`.

Basic Version

Approach 1 — structured output alone, no streaming yet

```
# models.py
from pydantic import BaseModel
```

```
class TicketResolution(BaseModel):
    category: str
    resolution_summary: str
    confidence: float
    needs_human_review: bool
```

```
# resolution.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from models import TicketResolution

RESOLUTION_PROMPT = ChatPromptTemplate.from_template(
    "Given this support ticket and the drafted reply, produce a resolution "
    "record: the category, a one-sentence resolution summary, your "
    "confidence from 0 to 1, and whether a human should review this before "
    "it's treated as resolved.\n\nTicket: {ticket}\nDrafted reply: {draft}"
)

def build_resolution_chain():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return RESOLUTION_PROMPT | model.with_structured_output(TicketResolution)
```

```
resolution_chain = build_resolution_chain()
result = resolution_chain.invoke({
    "ticket": "I was charged twice for my Pro subscription.",
    "draft": "I'm sorry about the duplicate charge -- I've flagged it for a refund."
})
print(result)
```

Expected output:

```
category='billing' resolution_summary='Duplicate charge flagged for refund.' confidence=0.95
```

This proves the structured half works, on its own, using a draft you typed in by hand — real streaming comes next.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — one draft, streamed and then summarized

```
# resolution.py -- add the streaming helper
def stream_draft(drafting_chain, ticket_input):
    for chunk in drafting_chain.stream(ticket_input):
        yield chunk
```

```
# main.py
from handlers import billing_chain
from resolution import build_resolution_chain, stream_draft

ticket_input = {"ticket": "I was charged twice for my Pro subscription, order INV-2201"}

print("Streaming draft to the UI:")
full_draft = ""
for piece in stream_draft(billing_chain, ticket_input):
    print(piece, end="", flush=True)
    full_draft += piece
print()

resolution_chain = build_resolution_chain()
resolution = resolution_chain.invoke({
    "ticket": ticket_input["ticket"],
    "draft": full_draft,
})
print("\nStructured record:")
print(resolution)
```

Expected output (the draft prints piece by piece, then the record):

```
Streaming draft to the UI:
I'm so sorry to hear about the duplicate charge on order INV-2201...

Structured record:
category='billing' resolution_summary='Duplicate charge on INV-2201, refund initi
```

Notice `full_draft` is built up out of the exact same pieces printed to the terminal — the resolution chain reads that, not a second, independently generated draft.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Approach 2 — carrying `needs_human_review` forward from earlier steps

```
# resolution.py -- a resolve() that combines the model's own judgment
# with the real signals Steps 1 and 3 already produced
def resolve_ticket(resolution_chain, ticket, draft, was_ambiguous, fallback_fired):
    result = resolution_chain.invoke({"ticket": ticket, "draft": draft})
    if was_ambiguous or fallback_fired:
        result.needs_human_review = True
    return result
```

Expected behavior: even if the resolution chain's own judgment says `needs_human_review=False`, a ticket that Step 1 flagged as genuinely ambiguous, or one where Step 3's fallback fired, still ends up flagged for a human — because those two signals are things you already know for a fact, not something worth leaving entirely up to a second model call's guess.

Difference from Basic: Basic proves the structured chain works on a hand-typed draft. Intermediate connects it to a real streamed draft from the actual drafting chain, and Approach 2 makes sure the two earlier steps' hard-won signals (ambiguity, fallback) don't get silently lost the moment Step 4 takes over.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — proving streaming is really incremental, not a fake typing effect

```
# test_pipeline.py (excerpt)
import time
from handlers import general_chain

ticket_input = {"ticket": "Do you have a referral program?"}

chunk_times = []
start = time.perf_counter()
for chunk in general_chain.stream(ticket_input):
    chunk_times.append(time.perf_counter() - start)

print(f"Received {len(chunk_times)} chunks")
print(f"First chunk at {chunk_times[0]:.2f}s, last at {chunk_times[-1]:.2f}s")

assert len(chunk_times) > 1, "Only one chunk arrived -- this isn't really streaming"
assert chunk_times[0] < chunk_times[-1], "All chunks arrived at once."
print("Confirmed: output is arriving incrementally, not all at once.")
```

Expected output:

```
Received 24 chunks
First chunk at 0.31s, last at 1.42s
Confirmed: output is arriving incrementally, not all at once.
```

If `len(chunk_times)` came back as `1`, that's a real red flag: something (a wrapping function, a `.invoke()` used by mistake instead of `.stream()`, or a fallback chain that doesn't support streaming) is collecting the whole answer before handing it back, defeating the entire point of this step.

Approach 2 — a fallback chain that can't stream, handled honestly

```
# resolution.py -- fallback chains built from RunnableLambda (Step 3's
# canned_response) don't naturally stream piece by piece; they just
# return one string. stream_draft() still works with them, it just
```

```
# yields exactly one chunk instead of many -- which is an honest
# reflection of reality: there's nothing progressive about a canned
# reply, so pretending to stream it word-by-word would be more
# misleading than just showing it all at once.

def stream_draft(drafting_chain, ticket_input):
    chunk_count = 0
    for chunk in drafting_chain.stream(ticket_input):
        chunk_count += 1
        yield chunk
    if chunk_count == 1:
        # Not necessarily a bug -- likely the fallback path fired,
        # which only ever produces one fixed piece of text.
        pass
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate connects real streaming to a real structured record, once. Approach 1 turns "it looks like it's streaming" into a real, automated check with actual chunk counts and timestamps. Approach 2 is about honesty under a real failure path: once Step 3's fallback fires, "streaming" a fixed canned string piece by piece would just be theater — a single chunk is the truthful representation of what actually happened, and pretending otherwise would hide, not reveal, that the fallback path ran.

Which one should you actually write? Approach 1's assertion-based chunk check, in your real test suite — it's the only way to know, later, if a refactor accidentally replaced `.stream()` with `.invoke()` somewhere and quietly broke the UX without breaking any functional test. Keep Approach 2's honest one-chunk behavior for the fallback path exactly as it is — don't build fake, artificial streaming for a canned reply just to make the two code paths look more similar than they actually are.